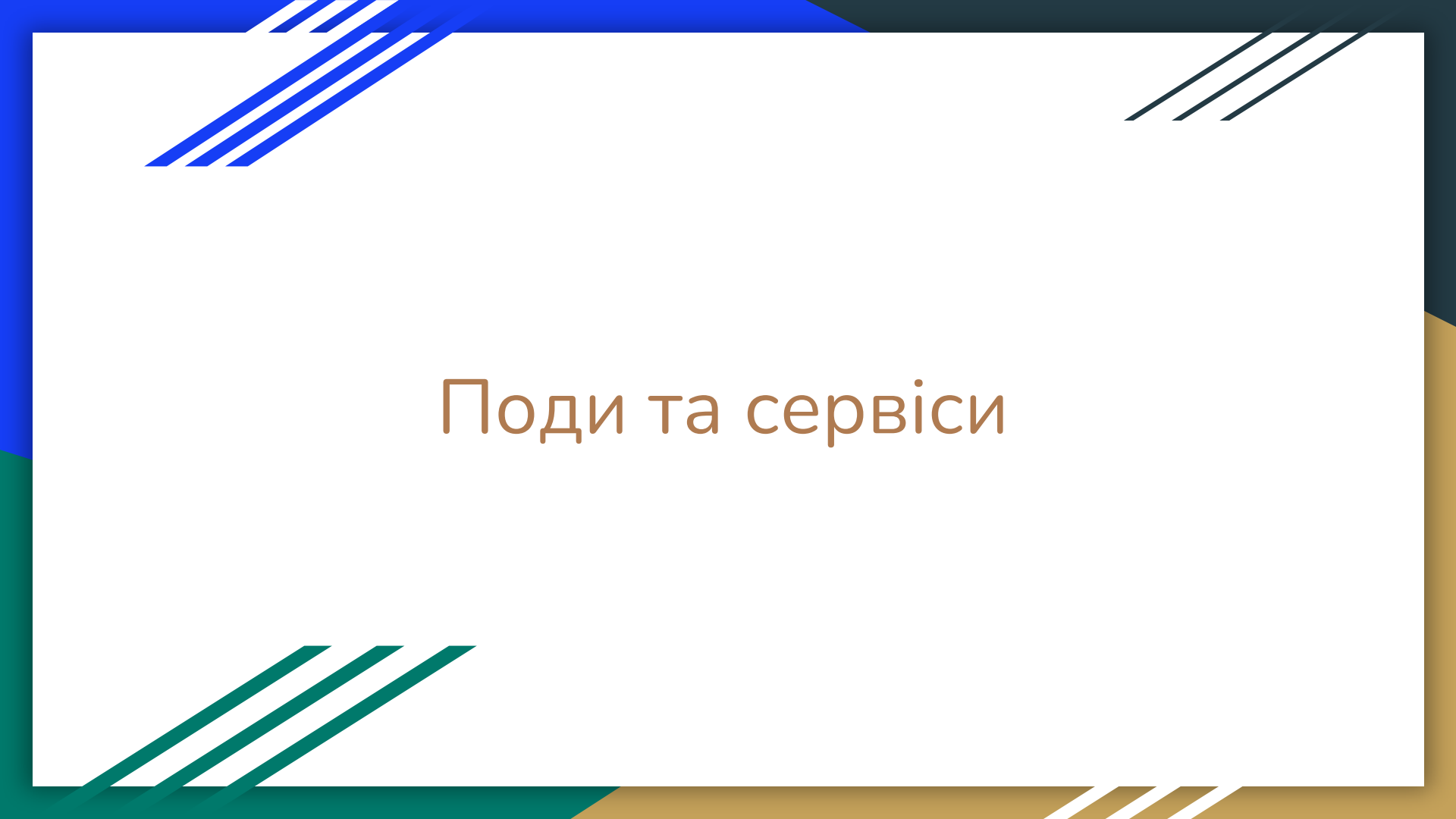




Поди та сервіси



Поди

Поды

Поды — це найменші розгорнуті обчислювальні одиниці, які можна створювати та керувати ними в Kubernetes.

Стручок (наприклад, стручок китів або гороховий стручок) — це група з одного або кількох контейнерів зі спільним сховищем і мережевими ресурсами, а також специфікацією того, як запускати контейнери. Вміст модуля завжди розміщується разом і за розкладом, а також працює в спільному контексті. Pod моделює специфічний для програми «логічний хост»: він містить один або більше контейнерів програми, які відносно тісно пов'язані. У нехмарних контекстах програми, які виконуються на одній фізичній або віртуальній машині, аналогічні хмарним програмам, які виконуються на тому самому логічному хості.

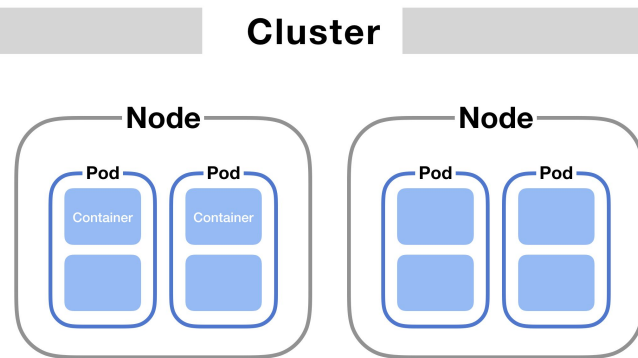
Окрім контейнерів додатків, Pod може містити контейнери ініціалізації, які запускаються під час запуску Pod. Ви також можете вставити ефемерні контейнери для налагодження, якщо ваш кластер це пропонує.

Що таке Pod?

Примітка. Незважаючи на те, що Kubernetes підтримує більше середовищ виконання контейнерів, ніж просто Docker, Docker є найпоширенішим середовищем виконання, і це допомагає описувати Pods, використовуючи певну термінологію Docker.

Спільний контекст Pod — це набір просторів імен Linux, контрольних груп і потенційно інших аспектів ізоляції — те саме, що ізолює контейнер Docker. У контексті Pod до окремих програм можуть застосовуватися додаткові субізоляції.

З точки зору концепції Docker, Pod схожий на групу Docker-контейнери зі спільними просторами імен і спільними томами файлової системи.



Використання Pods

Зазвичай вам не потрібно створювати модулі напряду, навіть одиночні пакети. Натомість створіть їх за допомогою ресурсів робочого навантаження, наприклад розгортання або завдання. Якщо вашим Pods потрібно відстежувати стан, розгляньте ресурс `StatefulSet`.

Поды в кластері Kubernetes використовуються двома основними способами:

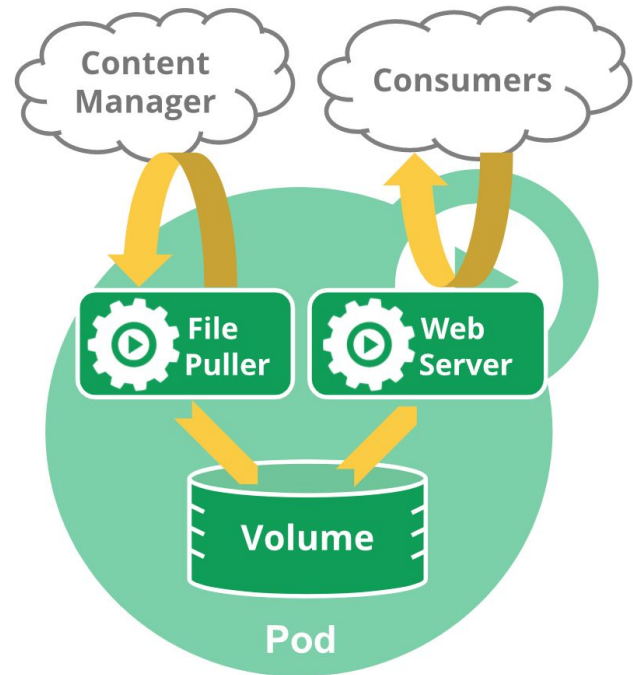
- **Поды, які працюють в одному контейнері.** Модель «один контейнер на под» є найпоширенішим випадком використання Kubernetes; у цьому випадку ви можете думати про Pod як обгортку навколо одного контейнера; Kubernetes керує Pods, а не безпосередньо контейнерами.
- **Поды, які запускають кілька контейнерів, які повинні працювати разом.** Pod може інкапсулювати додаток, що складається з кількох контейнерів, які тісно пов'язані між собою та потребують спільного використання ресурсів. Ці спільно розташовані контейнери утворюють єдину об'єднану одиницю служби — наприклад, один контейнер обслуговує дані, що зберігаються в спільному томі, для загального доступу, а окремий контейнер із колясками оновлює ці файли. Pod об'єднує ці контейнери, ресурси зберігання та ефемерну мережеву ідентифікацію в одне ціле.

Кожен Pod призначений для запуску окремого екземпляра певної програми. Якщо ви хочете горизонтально масштабувати свою програму (щоб забезпечити більше загальних ресурсів, запускаючи більше екземплярів), вам слід використовувати кілька модулів, по одному для кожного екземпляра. У Kubernetes це зазвичай називають реплікацією. Репліковані модулі зазвичай створюються та керуються як група ресурсом робочого навантаження та його контролером.

Як Pods керують кількома контейнерами

Блоки призначені для підтримки кількох взаємодіючих процесів (як контейнери), які утворюють єдину одиницю обслуговування. Контейнери в модулі автоматично розміщуються та спільно плануються на одній фізичній або віртуальній машині в кластері. Контейнери можуть ділитися ресурсами та залежностями, спілкуватися один з одним і координувати, коли та як вони припиняються.

Наприклад, у вас може бути контейнер, який виконує роль веб-сервера для файлів у спільному томі, і окремий контейнер, який оновлює ці файли з віддаленого джерела, як показано на наступній схемі:



Як Pods керують кількома контейнерами

Деякі Pods мають контейнери ініціалізації, а також контейнери програм. Контейнери ініціалізації запускаються та завершуються до запуску контейнерів програми.

Контейнери контейнерів надають два типи спільних ресурсів: мережеві та сховища.

Робота з Pods

Ви рідко створюватимете окремі модулі прямо в Kubernetes, навіть одиночні пакети. Це тому, що Pods розроблені як відносно ефемерні, одноразові об'єкти. Коли Pod створюється (безпосередньо вами або опосередковано контролером), новий Pod заплановано для запуску на Node у вашому кластері. Pod залишається на цьому вузлі, поки Pod не завершить виконання, об'єкт Pod не буде видалено, Pod не буде виселено через брак ресурсів або вузол не вийде з ладу.

Коли ви створюєте маніфест для об'єкта Pod, переконайтеся, що вказане ім'я є дійсним іменем субдомену DNS.

Поди та контролери

Ви можете використовувати ресурси робочого навантаження, щоб створювати та керувати кількома модулями. Контролер ресурсу керує реплікацією, розгортанням і автоматичним відновленням у разі збою модуля. Наприклад, якщо вузол виходить з ладу, контролер помічає, що модулі на цьому вузлі перестали працювати, і створює пакет на заміну. Планувальник поміщає заміну Pod на справний вузол.

Ось кілька прикладів ресурсів робочого навантаження, які керують одним або декількома модулями.

- Deployment
- StatefulSet
- DaemonSet

Шаблони подів

Контролери для ресурсів робочого навантаження створюють модулі з шаблону модуля та керують цими модулями від вашого імені.

PodTemplates — це специфікації для створення контейнерів, які входять до ресурсів робочого навантаження, таких як Deployments, Jobs і DaemonSets.

Кожен контролер для ресурсу робочого навантаження використовує PodTemplate всередині об'єкта робочого навантаження для створення фактичних модулів.

PodTemplate є частиною бажаного стану будь-якого ресурсу робочого навантаження, який ви використовували для запуску програми.

Зразок нижче є маніфестом для простого завдання з шаблоном, який запускає один контейнер. Контейнер у цьому модулі друкує повідомлення, а потім призупиняється.

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello
spec:
  template:
    # This is the pod template
    spec:
      containers:
        - name: hello
          image: busybox
          command: ['sh', '-c', 'echo "Hello, Kubernetes!" && sleep 3600']
          restartPolicy: OnFailure
    # The pod template ends here
```

Шаблони подів

Зміна шаблону модуля або перехід на новий шаблон пакета не має прямого впливу на модулі, які вже існують. Якщо ви змінюєте шаблон модуля для ресурсу робочого навантаження, цей ресурс має створити заміну модулів, які використовують оновлений шаблон.

Наприклад, контролер StatefulSet гарантує, що запущені модулі збігаються з поточним шаблоном модуля для кожного об'єкта StatefulSet. Якщо ви відредагуєте StatefulSet, щоб змінити його шаблон модуля, StatefulSet почне створювати нові модулі на основі оновленого шаблону. Згодом усі старі Pods буде замінено новими Pods, і оновлення завершено.

Кожен ресурс робочого навантаження реалізує власні правила обробки змін у шаблоні Pod. На вузлах kubelet безпосередньо не спостерігає та не керує будь-якими деталями навколо шаблонів і оновлень модулів; ці деталі абстрагуються. Така абстракція та розділення проблем спрощує семантику системи та робить можливим розширити поведінку кластера без зміни існуючого коду.

Оновлення та заміна Pod

Як згадувалося в попередньому розділі, коли змінюється шаблон Pod для ресурсу робочого навантаження, контролер створює нові Pod на основі оновленого шаблону замість оновлення або виправлення існуючих Pod.

Kubernetes не заважає вам безпосередньо керувати Pods. Можна оновити деякі поля запущеного модуля на місці. Однак операції оновлення Pod, такі як виправлення та заміна, мають деякі обмеження:

- Більшість метаданих про Pod є незмінними. Наприклад, ви не можете змінити простір імен, ім'я, uid або мітку часу створення; поле генерації унікальне. Він приймає лише оновлення, які збільшують поточне значення поля.
- Якщо встановлено `metadata.deletionTimestamp`, новий запис не можна додати до списку `metadata.finalizers`.
- Оновлення модулів не можуть змінювати поля, окрім `spec.containers[*].image`, `spec.initContainers[*].image`, `spec.activeDeadlineSeconds` або `spec.tolerations`. Для `spec.tolerations` ви можете додавати лише нові записи.
- Під час оновлення поля `spec.activeDeadlineSeconds` дозволено два типи оновлень:
 1. встановлення непризначеного поля додатного числа;
 2. оновлення поля з позитивного числа на менше, невід'ємне число.

Обмін ресурсами та спілкування

Блоки дозволяють обмінюватися даними та спілкуватися між їхніми складовими контейнерами.

Зберігання в контейнерах

Pod може вказати набір спільних томів сховища. Усі контейнери в Pod мають доступ до спільних томів, що дозволяє цим контейнерам обмінюватися даними. Томи також дозволяють постійним даним у модулі зберігатися, якщо один із контейнерів усередині потребує перезапуску.

Обмін ресурсами та спілкування

Pod мережа

Кожному модулю призначається унікальна IP-адреса для кожної групи адрес. Кожен контейнер у Pod спільно використовує мережевий простір імен, включаючи IP-адресу та мережеві порти. Усередині Pod (і тільки тоді) контейнери, які належать Pod, можуть спілкуватися один з одним за допомогою localhost. Коли контейнери в Pod обмінюються даними з об'єктами за межами Pod, вони повинні координувати, як вони використовують спільні мережеві ресурси (наприклад, порти). Усередині Pod контейнери спільно використовують IP-адресу та простір портів і можуть знаходити один одного через localhost. Контейнери в Pod також можуть спілкуватися один з одним за допомогою стандартних міжпроцесних комунікацій, таких як семафори SystemV або спільна пам'ять POSIX. Контейнери в різних модулях мають різні IP-адреси і не можуть обмінюватися даними через IPC без спеціального налаштування. Контейнери, які хочуть взаємодіяти з контейнером, що працює в іншому Pod, можуть використовувати IP-мережу для зв'язку.

Контейнери в модулі бачать ім'я системного хоста таким же, що й налаштоване ім'я модуля.

Привілейований режим для контейнерів

Будь-який контейнер у Pod може ввімкнути привілейований режим, використовуючи позначку привілейованого контексту безпеки специфікації контейнера. Це корисно для контейнерів, які хочуть використовувати адміністративні можливості операційної системи, такі як маніпулювання мережевим стеком або доступ до апаратних пристроїв. Процеси всередині привілейованого контейнера отримують майже ті самі привілеї, які доступні для процесів поза контейнером.

Статичні поди

Static Pods керуються безпосередньо демоном kubelet на певному вузлі без спостереження сервера API. У той час як більшістю модулів керує контрольна площа (наприклад, розгортання), для статичних модулів kubelet безпосередньо контролює кожен статичний пакет (і перезапускає його, якщо він виходить з ладу).

Статичні модулі завжди прив'язані до одного Kubelet на певному вузлі. Основне використання статичних модулів — це запуск самостійної площини керування: іншими словами, використання kubelet для нагляду за окремими компонентами площини керування.

Kubelet автоматично намагається створити дзеркало Pod на сервері Kubernetes API для кожного статичного Pod. Це означає, що модулі, запущені на вузлі, видимі на сервері API, але ними не можна керувати звідти.



Життєвий цикл Pod

Pod фаза

Поле статусу Pod — це об'єкт PodStatus, який має поле фази.

Фаза Pod — це простий підсумок високого рівня того, на якому етапі життєвого циклу знаходиться Pod. Ця фаза не призначена як комплексне зведення спостережень за станом контейнера чи контейнера, а також як комплексна машина стану.

Кількість і значення значень фази Pod суворо охороняються. Крім того, що тут задокументовано, нічого не слід припускати щодо модулів, які мають задане значення фази. Якщо вузол вимикається або від'єднується від решти кластера, Kubernetes застосовує політику для налаштування фази всіх модулів на втраченому вузлі на «Збій».

Ось можливі значення для фази:

Value	Description
-------	-------------

Pending	The Pod has been accepted by the Kubernetes cluster, but one or more of the containers has not been set up and made ready to run. This includes time a Pod spends waiting to be scheduled as well as the time spent downloading container images over the network.
---------	--

Running	The Pod has been bound to a node, and all of the containers have been created. At least one container is still running, or is in the process of starting or restarting.
---------	---

Successful	All containers in the Pod have terminated in success, and will not be restarted.
------------	--

Failed	All containers in the Pod have terminated, and at least one container has terminated in failure. That is, the container either exited with non-zero status or was terminated by the system.
--------	---

Unknown	For some reason the state of the Pod could not be obtained. This phase typically occurs due to an error in communicating with the node where the Pod should be running.
---------	---

Стан контейнера

Окрім загальної фази Pod, Kubernetes відстежує стан кожного контейнера всередині Pod. Ви можете використовувати хуки життєвого циклу контейнера, щоб запускати події, які запускаються в певні моменти життєвого циклу контейнера.

Коли планувальник призначає Pod вузлу, kubelet починає створювати контейнери для цього Pod за допомогою середовища виконання контейнера. Є три можливі стани контейнера: Очікування, Виконується та Припинено.

Щоб перевірити стан контейнерів Pod, ви можете використати `kubectl describe pod <name-of-pod>`. Вихідні дані показують стан кожного контейнера в цьому модулі.

Кожен стан має певне значення:

Очікування

Якщо контейнер не перебуває ні в стані Running, ні в статусі Terminated, він знаходиться в очікуванні. Контейнер у стані очікування все ще виконує операції, необхідні для завершення запуску: наприклад, отримання зображення контейнера з реєстру зображень контейнера або застосування секретних даних. Коли ви використовуєте `kubectl` для запиту Pod з контейнером, який очікує, ви також бачите поле Reason, щоб підсумувати, чому контейнер знаходиться в такому стані.

Стан контейнера

Виконується

Статус `Running` вказує на те, що контейнер виконується без проблем. Якщо був налаштований хук `postStart`, він уже виконано та завершено. Коли ви використовуєте `kubectl` для запиту Pod із контейнером, який працює, ви також бачите інформацію про те, коли контейнер перейшов у стан `Running`.

Припинено

Контейнер у стані "Припинено" почав виконання, а потім або працював до завершення, або з певної причини вийшов з ладу. Коли ви використовуєте `kubectl` для запиту Pod із закритим контейнером, ви бачите причину, код виходу, а також час початку та завершення періоду виконання цього контейнера.

Якщо контейнер має налаштований хук `preStop`, він запускається до того, як контейнер перейде в стан `Terminated`.

Політика перезавпуску контейнера

Специфікація Pod містить поле `restartPolicy` із можливими значеннями `Always`, `OnFailure` та `Never`. Значення за замовчуванням — завжди.

Політика перезавпуску застосовується до всіх контейнерів у модулі. `restartPolicy` стосується лише перезавпусків контейнерів kubelet на тому самому вузлі. Після виходу контейнерів у Pod kubelet перезавпускає їх із експоненціальною затримкою (10 с, 20 с, 40 с, ...), яка обмежена п'ятьма хвилинами. Після того, як контейнер виконувався протягом 10 хвилин без проблем, kubelet скидає таймер відстрочки перезавпуску для цього контейнера.

Перевірки контейнера

Кожен зонд має один із трьох результатів:

- Успіх : контейнер пройшов діагностику.
- Помилка : контейнер не пройшов діагностику.
- Невідомо : не вдалося виконати діагностику, тому не потрібно вживати жодних дій.

Pod умови

Pod має PodStatus, який містить масив PodConditions, через які Pod пройшов або не пройшов:

- **PodScheduled**: модуль було заплановано для вузла.
- **ContainersReady**: всі контейнери в Pod готові.
- **Initialized**: усі контейнери ініціалізації запущено успішно.
- **Ready**: Pod може обслуговувати запити, і його слід додати до пулів балансування навантаження всіх відповідних Сервісів.

Field name	Description
type	Name of this Pod condition.
status	Indicates whether that condition is applicable, with possible values " True ", " False ", Or " Unknown ".
lastProbeTime	Timestamp of when the Pod condition was last probed.
lastTransitionTime	Timestamp for when the Pod last transitioned from one status to another.
reason	Machine-readable, UpperCamelCase text indicating the reason for the condition's last transition.
message	Human-readable message indicating details about the last status transition.

Перевірки контейнера

Kubelet може додатково виконувати та реагувати на три види зондів на запущених контейнерах:

- `livenessProbe`: вказує, чи запущений контейнер. Якщо тест живучості не вдається, kubelet вбиває контейнер, і контейнер піддається своїй політиці перезапуску. Якщо Контейнер не забезпечує перевірку живучості, станом за умовчанням є Успіх.
- `readinessProbe`: вказує, чи готовий контейнер відповідати на запити. Якщо перевірка готовності не вдається, контролер кінцевих точок видаляє IP-адресу модуля з кінцевих точок усіх служб, які відповідають модулю. Стандартним станом готовності до початкової затримки є Помилка. Якщо контейнер не забезпечує перевірку готовності, станом за замовчуванням є успіх.
- `startupProbe`: вказує, чи запущено програму всередині контейнера. Усі інші тести вимикаються, якщо надається тест запуску, доки він не буде успішним. Якщо тест запуску не вдається, kubelet вимикає контейнер, і контейнер підлягає своїй політиці перезапуску. Якщо контейнер не надає зонд запуску, стан за замовчуванням – успішно.

Коли слід використовувати зонд Liveness?

СТАН ФУНКЦІЇ: Kubernetes v1.0 [стабільна]

Якщо процес у вашому контейнері здатний аварійно завершувати роботу щоразу, коли виникає проблема або стає несправним, вам не обов'язково потрібен зонд живучості; kubelet автоматично виконає правильну дію згідно з політикою перезапуску модуля.

Якщо ви хочете, щоб ваш контейнер був закритий і перезапущений у разі збою тестування, тоді вкажіть зонд живучості та вкажіть для параметра `restartPolicy` значення `Always` або `OnFailure`.

Коли слід використовувати зонд готовності?

СТАН ФУНКЦІЇ: `Kubernetes v1.0` [стабільна]

Якщо ви хочете розпочати надсилання трафіку до Pod лише після успішного тестування, укажіть тестування готовності. У цьому випадку тест готовності може бути таким самим, як і тест живучості, але наявність тесту готовності в специфікації означає, що Pod запусниться, не отримуючи жодного трафіку, і почне отримувати трафік лише після того, як тест почне успішно працювати. Якщо під час запуску ваш контейнер повинен працювати над завантаженням великих даних, файлів конфігурації або міграцій, укажіть зонд готовності.

Якщо ви хочете, щоб ваш контейнер міг самостійно зніматися для обслуговування, ви можете вказати зонд живучості, який перевіряє кінцеву точку, специфічну для живучості, яка відрізняється від тесту живучості.

Коли слід використовувати зонд запуску?

СТАН ФУНКЦІЇ: Kubernetes v1.20 [стабільна]

Зонди запуску корисні для модулів, які мають контейнери, які довго вводяться в експлуатацію. Замість того, щоб встановлювати тривалий інтервал живучості, ви можете налаштувати окрему конфігурацію для тестування контейнера під час його запуску, дозволяючи час, довший за інтервал живучості.

Якщо ваш контейнер зазвичай запускається за більше ніж `initialDelaySeconds` + `failureThreshold` × `periodSeconds`, ви повинні вказати зонд запуску, який перевіряє ту саму кінцеву точку, що й тест живучості. Типовим значенням для `periodSeconds` є 10 секунд. Потім ви повинні встановити для параметра `failureThreshold` достатньо високий рівень, щоб дозволити контейнеру запуснитися, не змінюючи стандартні значення зонда живучості. Це допомагає захиститися від взаємоблокувань.

Припинення Pod

Оскільки модулі представляють процеси, що виконуються на вузлах у кластері, важливо дозволити цим процесам витончено завершуватись, коли вони більше не потрібні (замість того, щоб раптово зупинятися за допомогою сигналу KILL і не мати можливості очистити).

Мета дизайну полягає в тому, щоб ви могли запитувати видалення та знати, коли процеси завершуються, а також мати можливість гарантувати, що видалення врешті-решт завершено. Коли ви надсилаєте запит на видалення Pod, кластер записує та відстежує передбачуваний пільговий період перед тим, як буде дозволено примусово знищити Pod. За допомогою відстеження примусового завершення роботи kubelet намагається плавно завершити роботу.

Як правило, середовище виконання контейнера надсилає сигнал TERM головному процесу в кожному контейнері. Багато середовищ виконання контейнерів поважають значення STOPSIGNAL, визначене в образі контейнера, і надсилають його замість TERM. Після закінчення пільгового періоду сигнал KILL надсилається до будь-яких процесів, що залишилися, а Pod видаляється з сервера API. Якщо kubelet або службу керування середовищем виконання контейнера перезапущається під час очікування завершення процесів, кластер повторює спробу з початку, включаючи повний початковий пільговий період.

Припинення Pod

Приклад потоку:

1. Ви використовуєте інструмент `kubectl`, щоб вручну видалити певний модуль із пільговим періодом за замовчуванням (30 секунд).
2. Под на сервері API оновлюється час, після якого модуль вважається «мертвим», а також пільговий період. Якщо ви використовуєте `kubectl describe` для перевірки модуля, який ви видаляєте, цей модуль відображається як «Завершення». На вузлі, де запущено Pod: щойно kubelet бачить, що Pod було позначено як завершення (встановлено тривалість витонченого завершення роботи), kubelet починає локальний процес завершення роботи Pod.
 1. Якщо один із контейнерів Pod визначив хук `preStop`, kubelet запускає цей хук усередині контейнера. Якщо хук `preStop` продовжує працювати після закінчення пільгового періоду, kubelet запитує невелике одноразове продовження пільгового періоду на 2 секунди.
 2. Kubelet ініціює середовище виконання контейнера для надсилання сигналу `TERM` для обробки 1 у кожному контейнері.
3. У той самий час, коли kubelet починає плавне завершення роботи, площа керування видаляє цей модуль вимикання з об'єктів `Endpoints` (і, якщо ввімкнено, `EndpointSlice`), де вони представляють службу з налаштованим селектором. `ReplicaSets` та інші ресурси робочого навантаження більше не розглядають модуль, що вимикається, як дійсну копію, що працює в службі. Модулі, які повільно вимикаються, не можуть продовжувати обслуговувати трафік, оскільки балансувальники навантаження (наприклад, проксі-сервер служби) видаляють модуль зі списку кінцевих точок, щойно починається пільговий період припинення.

Припинення Pod

4. Коли пільговий період закінчується, kubelet запускає примусове завершення роботи. Середовище виконання контейнера надсилає SIGKILL будь-яким процесам, які все ще виконуються в будь-якому контейнері в модулі. Kubelet також очищає прихований контейнер паузи, якщо його використовує середовище виконання контейнера.
5. Kubelet ініціює примусове видалення об'єкта Pod із сервера API, установивши пільговий період на 0 (негайне видалення).
6. Сервер API видаляє об'єкт API Pod, який потім більше не відображається жодним клієнтом.

Примусове завершення Pod

За замовчуванням усі видалення виконуються протягом 30 секунд. Команда видалення `kubectl` підтримує параметр `--grace-period=<seconds>`, який дозволяє замінити значення за замовчуванням і вказати власне значення.

Встановлення пільгового періоду на 0 примусово та негайно видаляє Pod із сервера API. Якщо модуль все ще працював на вузлі, це примусове видалення ініціює kubelet, щоб розпочати негайне очищення.

Коли виконується примусове видалення, сервер API не чекає підтвердження від kubelet про те, що Pod було припинено на вузлі, на якому він працював. Він негайно видаляє Pod в API, щоб можна було створити новий Pod з такою ж назвою. На вузлі модулі, які налаштовані на негайне припинення роботи, отримують невеликий пільговий період перед примусовим знищенням.

Збір сміття невдалих Pods

Для невдалих Pods об'єкти API залишаються в API кластера, доки процес людини або контролера явно не видалить їх.

Площина керування очищає припинені модулі (з фазою «Успішно» або «Не вдалося»), коли кількість модулів перевищує налаштований поріг (визначений порогом `terminated-pod-gc-threshold` у `kube-controller-manager`). Це дозволяє уникнути витоку ресурсів, оскільки контейнери з часом створюються та припиняються.



Сервіс

Сервіс

Абстрактний спосіб представити програму, що працює на наборі Pods, як мережеву службу.

З Kubernetes вам не потрібно змінювати свою програму, щоб використовувати механізм виявлення незнайомих служб. Kubernetes надає Pods власні IP-адреси та єдине DNS-ім'я для набору Pods і може балансувати навантаження між ними.

Сервісні ресурси

У Kubernetes служба — це абстракція, яка визначає логічний набір контейнерів і політику доступу до них (іноді цей шаблон називають мікросервісом). Набір модулів, на які націлена служба, зазвичай визначається селектором. Наприклад, розглянемо серверну програму обробки зображень без стану, яка працює з 3 репліками. Ці репліки взаємозамінні — інтерфейсам байдуже, який сервер вони використовують. Хоча фактичні модулі, які складають набір серверних модулів, можуть змінюватися, клієнти зовнішнього інтерфейсу не повинні знати про це, а також їм не потрібно самостійно стежити за набором серверних модулів. Абстракція служби дозволяє це відокремити.

Виявлення хмарних сервісів

Якщо ви можете використовувати Kubernetes API для виявлення служб у своїй програмі, ви можете запитати сервер API для кінцевих точок, які оновлюються щоразу, коли змінюється набір модулів у службі.

Для невласних програм Kubernetes пропонує способи розмістити мережевий порт або балансир навантаження між вашою програмою та серверними панелями.

Визначення служби

Сервіс у Kubernetes — це об'єкт REST, схожий на Pod. Як і всі об'єкти REST, ви можете ПУСТИТИ визначення служби на сервер API, щоб створити новий екземпляр. Ім'я об'єкта служби має бути дійсним ім'ям мітки DNS.

Наприклад, припустимо, що у вас є набір модулів, кожен з яких слухає TCP-порт 9376 і має мітку

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Визначення служби

Ця специфікація створює новий об'єкт служби під назвою «my-service», який націлений на TCP-порт 9376 на будь-якому модулі з міткою app=MyApp.

Kubernetes призначає цій Службі IP-адресу (іноді її називають «IP-адресою кластера»), яка використовується проксі-серверами Служби.

Контролер для селектора служби постійно шукає модулі, які відповідають його селектору, а потім розміщує будь-які оновлення в об'єкті кінцевої точки, який також називається «моя служба».

Визначення портів у Pods мають імена, і ви можете посилатися на ці імена в атрибуті targetPort служби. Це працює, навіть якщо в службі є суміш модулів, які використовують одне налаштоване ім'я та той самий мережевий протокол, доступний через різні номери портів. Це забезпечує велику гнучкість для розгортання та розвитку ваших Послуг. Наприклад, ви можете змінити номери портів, які Pods надають у наступній версії вашого серверного програмного забезпечення, не порушуючи роботу клієнтів. Стандартним протоколом для Служб є TCP. Оскільки багатьом службам потрібно надавати більше ніж один порт, Kubernetes підтримує кілька визначень портів для об'єкта служби. Кожне визначення порту може мати той самий протокол або інший.

Сервіси без селекторів

Сервіси найчастіше абстрагують доступ до Kubernetes Pods, але вони також можуть абстрагувати інші види серверних програм. Наприклад:

- Ви хочете мати зовнішній кластер бази даних у виробництві, але у вашому тестовому середовищі ви використовуєте власні бази даних.
- Ви хочете вказати свою службу на службу в іншому просторі імен або в іншому кластері.
- Ви переносите робоче навантаження в Kubernetes. Оцінюючи підхід, ви запускаєте лише частину своїх серверних програм у Kubernetes.

У будь-якому з цих сценаріїв ви можете визначити службу без селектора Pod. Наприклад:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
```

Сервіси без селекторів

Оскільки ця служба не має селектора, відповідний об'єкт Endpoints не створюється автоматично. Ви можете вручну зіставити службу з мережевою адресою та портом, де вона працює, додавши об'єкт Endpoints вручну:

Ім'я об'єкта Endpoints має бути дійсним іменем субдомену DNS.

Доступ до послуги без селектора працює так само, якби він мав селектор. У прикладі вище трафік направляється до єдиної кінцевої точки, визначеної в YAML: 192.0.2.42:9376 (TCP).

Служба зовнішніх імен — це окремий випадок служби, яка не має селекторів і замість них використовує імена DNS.

```
apiVersion: v1
kind: Endpoints
metadata:
  name: my-service
subsets:
  - addresses:
      - ip: 192.0.2.42
    ports:
      - port: 9376
```

EndpointSlices

СТАН ФУНКЦІЙ: Kubernetes v1.17 [бета]

EndpointSlices — це ресурс API, який може надати більш масштабовану альтернативу кінцевим точкам. Хоча концептуально вони досить схожі на Endpoints, EndpointSlices дозволяють розподіляти кінцеві точки мережі між кількома ресурсами. За замовчуванням EndpointSlice вважається «повним», коли він досягає 100 кінцевих точок, після чого будуть створені додаткові EndpointSlice для зберігання будь-яких додаткових кінцевих точок.

EndpointSlices надають додаткові атрибути та функції, які детально описано в EndpointSlices.

Протокол застосування

СТАН ФУНКЦІЇ: Kubernetes v1.20 [стабільна]

Поле `appProtocol` надає спосіб вказати протокол програми для кожного порту служби. Значення цього поля відображається відповідними об'єктами `Endpoints` і `EndpointSlice`.

Це поле відповідає стандартному синтаксису міток Kubernetes. Значення мають бути стандартними іменами служб IANA або іменами з префіксом домену, наприклад `mycompany.com/my-custom-protocol`.

Віртуальні IP-адреси та сервісні проксі

Кожен вузол у кластері Kubernetes запускає kube-проксі. kube-proxy відповідає за реалізацію форми віртуальної IP-адреси для Сервісів іншого типу, ніж ExternalName.

Чому б не використати циклічний DNS?

Час від часу виникає питання, чому Kubernetes покладається на проксі-сервер для пересилання вхідного трафіку на серверні частини. А як щодо інших підходів? Наприклад, чи можливо було б налаштувати DNS-записи, які мають кілька значень A (або AAAA для IPv6), і покладатися на циклічне розпізнавання імен?

Є кілька причин для використання проксі для Служб:

- Існує довга історія реалізацій DNS, які не поважають TTL записів і кешують результати пошуку імен після того, як вони повинні були закінчитися.
- Деякі програми виконують DNS-пошук лише один раз і кешують результати на невизначений термін.
- Навіть якщо програми та бібліотеки виконали належне повторне вирішення, низькі або нульові значення TTL у записах DNS можуть створювати велике навантаження на DNS, яким стає важко керувати.

Режим проксі-сервера простору користувача

У цьому режимі kube-proxy спостерігає за рівнем керування Kubernetes для додавання та видалення об'єктів Service і Endpoint. Для кожної служби відкривається порт (вибраний випадковим чином) на локальному вузлі. Будь-які підключення до цього «проксі-порту» надсилаються через проксі до одного з серверних модулів служби (як повідомляється через кінцеві точки). kube-proxy бере до уваги налаштування SessionAffinity Служби, коли вирішує, який серверний Pod використовувати.

Нарешті, проксі-сервер простору користувача встановлює правила iptables, які перехоплюють трафік до IP-адреси кластера Служби (який є віртуальним) і порту. Правила перенаправляють цей трафік на проксі-порт, який є проксі-сервером сервера Pod.

За замовчуванням kube-proxy у режимі простору користувача вибирає серверну частину за допомогою циклічного алгоритму.

Багатопортові служби

Для деяких служб потрібно відкрити більше одного порту. Kubernetes дозволяє налаштувати кілька визначень портів для об'єкта служби. Використовуючи кілька портів для служби, ви повинні вказати всі назви своїх портів, щоб вони були однозначними. Наприклад:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
    - name: https
      protocol: TCP
      port: 443
      targetPort: 9377
```

Вибір власної IP-адреси

Ви можете вказати власну IP-адресу кластера як частину запиту на створення служби. Для цього встановіть поле `.spec.clusterIP`. Наприклад, якщо у вас уже є запис DNS, який ви бажаєте повторно використати, або застарілі системи, налаштовані на конкретну IP-адресу, які важко переналаштувати.

Вибрана вами IP-адреса має бути дійсною адресою IPv4 або IPv6 із діапазону CIDR кластера служби, налаштованого для сервера API. Якщо ви спробуєте створити службу з недійсним значенням IP-адреси кластера, сервер API поверне код статусу HTTP 422, щоб вказати, що виникла проблема.

Виявлення служб

Kubernetes підтримує 2 основні режими пошуку служби – змінні середовища та DNS.

Змінні середовища

Коли Pod запускається на вузлі, kubelet додає набір змінних середовища для кожної активної служби. Він підтримує як змінні, сумісні з посиланнями Docker, так і простіші змінні {SVCNAME}_SERVICE_HOST і {SVCNAME}_SERVICE_PORT, де назва служби пишеться у верхньому регістрі, а тире перетворюються на підкреслення.

Наприклад, служба redis-master, яка надає доступ до порту TCP 6379 і якій призначено IP-адресу кластера 10.0.0.11, створює такі змінні середовища:

```
REDIS_MASTER_SERVICE_HOST=10.0.0.11
REDIS_MASTER_SERVICE_PORT=6379
REDIS_MASTER_PORT=tcp://10.0.0.11:6379
REDIS_MASTER_PORT_6379_TCP=tcp://10.0.0.11:6379
REDIS_MASTER_PORT_6379_TCP_PROTO=tcp
REDIS_MASTER_PORT_6379_TCP_PORT=6379
REDIS_MASTER_PORT_6379_TCP_ADDR=10.0.0.11
```

Публікаційні служби (ServiceTypes)

Для деяких частин вашої програми (наприклад, зовнішніх інтерфейсів) ви можете надати Службу зовнішній IP-адресі, яка знаходиться за межами вашого кластера.

Kubernetes `ServiceTypes` дає змогу вказати, яку службу ви хочете отримати. За замовчуванням `ClusterIP`.

Значення типу та їх поведінка:

- `ClusterIP`: Розкриває службу на внутрішньому IP-адресі кластера. Вибір цього значення робить службу доступною лише з кластера. Це типовий тип служби.
- `NodePort`: Відкриває службу на IP-адресі кожного вузла на статичному порту (`NodePort`). Служба `ClusterIP`, до якої маршрутизується служба `NodePort`, створюється автоматично. Ви зможете зв'язатися зі службою `NodePort` поза межами кластера, запитавши `<NodeIP>:<NodePort>`.
- `LoadBalancer`: Надає Службу зовнішньому доступу за допомогою балансувальника навантаження хмарного постачальника. Служби `NodePort` і `ClusterIP`, до яких маршрутизує зовнішній балансувальник навантаження, створюються автоматично.
- `ExternalName`: Зіставляє службу з вмістом поля `externalName` (наприклад, `foo.bar.example.com`), повертаючи запис CNAME із його значенням. Жоден проксі-сервер не встановлено.

Типи NodePort

Якщо ви встановите для поля типу значення NodePort, площина керування Kubernetes виділяє порт із діапазону, визначеного прапором `--service-node-port-range` (за замовчуванням: 30000-32767). Кожен вузол передає цей порт (однаковий номер порту на кожному вузлі) у ваш Сервіс. Ваша служба повідомляє про виділений порт у своєму полі `.spec.ports[*].nodePort`.

Якщо ви хочете вказати конкретні IP-адреси для проксі-сервера порту, ви можете встановити прапорець `--nodeport-addresses` у kube-proxy для певних IP-блоків; це підтримується з Kubernetes v1.10. Цей прапорець використовує список IP-блоків, розділених комами (наприклад, 10.0.0.0/8, 192.0.2.0/25), щоб визначити діапазони IP-адрес, які kube-proxy має вважати локальними для цього вузла.

Наприклад, якщо ви запускаєте kube-proxy із прапорцем `--nodeport-addresses=127.0.0.0/8`, kube-proxy вибирає лише інтерфейс петлі для служб NodePort. За замовчуванням для `--nodeport-addresses` є порожній список. Це означає, що kube-proxy має враховувати всі доступні мережеві інтерфейси для NodePort. (Це також сумісно з попередніми випусками Kubernetes).

Якщо вам потрібен певний номер порту, ви можете вказати значення в полі `nodePort`. Площина керування або виділить вам цей порт, або повідомить про помилку транзакції API. Це означає, що ви повинні самостійно подбати про можливі зіткнення портів. Ви також повинні використовувати дійсний номер порту, який знаходиться в межах діапазону, налаштованого для використання NodePort.

Типи NodePort

Використання NodePort дає вам свободу налаштувати власне рішення балансування навантаження, налаштувати середовища, які не повністю підтримуються Kubernetes, або навіть просто відкрити IP-адреси одного або кількох вузлів безпосередньо.

Зауважте, що ця служба відображається як `<NodeIP>:spec.ports[*].nodePort i .spec.clusterIP:spec.ports[*].port`. (Якщо прапорець `--nodeport-addresses` у kube-proxy встановлено, NodeIP буде відфільтровано).

Наприклад:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app: MyApp
  ports:
    # By default and for convenience, the `targetPort` is set to the same value as the `port` field.
    - port: 80
      targetPort: 80
    # Optional field
    # By default and for convenience, the Kubernetes control plane will allocate a port from a range (default: 30000-32767)
    nodePort: 30007
```

Тип LoadBalancer

У хмарних провайдерах, які підтримують зовнішні балансувальники навантаження, налаштування поля типу `LoadBalancer` забезпечує балансування навантаження для вашої Служби. Фактичне створення балансувальника навантаження відбувається асинхронно, а інформація про наданий балансувальник публікується в полі `.status.loadBalancer` служби. Наприклад:

Трафік із зовнішнього балансувальника навантаження спрямовується на серверні модулі. Хмарний постачальник вирішує, як розподіляти навантаження.

Деякі хмарні постачальники дозволяють вказати `loadBalancerIP`. У таких випадках балансувальник навантаження створюється із зазначеним користувачем IP-адресою `loadBalancer`. Якщо поле `loadBalancerIP` не вказано, `loadBalancer` налаштовано з ефемерною IP-адресою. Якщо ви вказали `loadBalancerIP`, але ваш хмарний постачальник не підтримує цю функцію, поле `loadBalancerIP`, яке ви налаштували, ігноруватиметься.

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - protocol: TCP
      port: 80
      targetPort: 9376
  clusterIP: 10.0.171.239
  type: LoadBalancer
status:
  loadBalancer:
    ingress:
      - ip: 192.0.2.127
```

Тип ExternalName

Служби типу ExternalName відображають службу на ім'я DNS, а не на типовий селектор, такий як my-service або cassandra. Ви вказуєте ці служби за допомогою параметра spec.externalName.

Це визначення служби, наприклад, відображає службу my-service у просторі імен prod на my.database.example.com:

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

Тип ExternalName

Під час пошуку хосту `my-service.prod.svc.cluster.local` служба DNS кластера повертає запис CNAME зі значенням `my.database.example.com`. Доступ до моєї служби працює так само, як і інші служби, але з тією важливою різницею, що перенаправлення відбувається на рівні DNS, а не через проксі-сервер або переадресацію. Якщо пізніше ви вирішите перемістити свою базу даних у кластер, ви можете запустити її модулі, додати відповідні селектори чи кінцеві точки та змінити тип служби.

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName: my.database.example.com
```

Зовнішні IP-адреси

Якщо є зовнішні IP-адреси, які направляють до одного або кількох вузлів кластера, служби Kubernetes можуть бути доступні для цих зовнішніх IP-адрес. Трафік, який надходить у кластер із зовнішньою IP-адресою (як IP-адресою призначення) на порту служби, буде направлено до однієї з кінцевих точок служби. зовнішні IP-адреси не керуються Kubernetes і за них відповідає адміністратор кластера.

У специфікації служби можна вказати зовнішні IP-адреси разом із будь-яким типом служби. У наведеному нижче прикладі клієнти можуть отримати доступ до «my-service» на «80.11.12.10:80» (externalIP:port)

```
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: MyApp
  ports:
    - name: http
      protocol: TCP
      port: 80
      targetPort: 9376
  externalIPs:
    - 80.11.12.10
```

Питання та відповіді